SOFTWARE REQUIREMENTS SPECIFICATION

FinPay Core Engine

Part 2 | RESTful Web API | ASP.NET Core, EF Core & SQL Server

Project	FinPay Core Engine
Purpose	Define the proposed architecture, data model, API endpoints, implementation concepts, and unit-testing scope.
Target Stack	C#, ASP.NET Core Web API, Entity Framework Core, SQL Server, xUnit, and Moq

2.1 System Architecture and Folder Structure

The solution will be organized into four layers to support **SOLID principles**, separation of concerns, and **loose coupling**. Dependencies should point inward: the Domain layer should not depend on infrastructure or API details.

Layer / Project	Responsibilities	Typical Contents
FinPay.Domain	Core domain model and OOP	Entities such as `User`, `Wallet`, and `Transaction`; enums such as `TransactionType` and `WalletStatus`; abstract classes.
FinPay.Application	Use cases, interfaces, DTOs, and business logic	`IWalletService`, `ITransferService`, `TransferRequestDto`, and custom exceptions such as `InsufficientBalanceException`.
FinPay.Infrastructure	Persistence and external implementation details	`AppDbContext`, EF Core migrations, SQL Server configuration, and repository implementations such as `WalletRepository`.
FinPay.API	HTTP interface and application composition	Controllers, `Program.cs` dependency-injection setup, JWT authentication, authorization, and global exception-handling middleware.

2.2 Database Schema — SQL Server and Entity Framework Core

The database will be created using the **Code-First** approach. EF Core entity configurations and migrations will be used to create and evolve the schema.

Table	Columns and Data Types	EF Core / SQL Concepts
Users	`Id` (PK, Guid); `FullName` (nvarchar); `Email` (unique index); `PhoneNumber` (unique index); `PasswordHash` (string); `Role` (Customer/Admin enum); `CreatedAt` (DateTime).	One-to-one relationship with `Wallets`. Unique indexes on email and phone number support uniqueness and lookup.
Wallets	`Id` (PK, Guid); `UserId` (FK); `WalletNumber` (unique, 10 digits); `Balance` (decimal(18,2)); `WalletType` (Basic/Merchant enum); `IsFrozen` (bool); `RowVersion` (byte[]).	`[Timestamp]` or `IsRowVersion()` for optimistic concurrency, helping prevent conflicting simultaneous balance updates.
Transactions	`Id` (PK, Guid); `ReferenceId` (unique string); `SenderWalletId` (nullable FK); `ReceiverWalletId` (FK); `Amount` (decimal(18,2)); `Fee` (decimal(18,2)); `Status` (Pending/Completed/Failed enum); `CreatedAt` (DateTime).	Relationships to wallets and ledger entries. Configure Fluent API delete behavior as `Restrict` so wallet deletion does not erase transaction history.
LedgerEntries	`Id` (PK, long); `TransactionId` (FK); `WalletId` (FK); `EntryType` (Debit/Credit enum); `Amount` (decimal); `BalanceAfter` (decimal); `Timestamp` (DateTime).	Append-only audit trail. Each successful P2P transfer creates two ledger rows: one debit and one credit.

Implementation note: the ledger should be treated as an audit record and should not be edited or deleted during normal application operations. Database transactions and concurrency checks must protect wallet balances and ledger consistency.

2.3 RESTful API Endpoints

Endpoints below define the initial API surface. Authenticated routes require a valid JWT unless explicitly marked Public. Admin routes require the Admin role.

Module	Verb	Route	Access	Purpose / Technical Notes
Auth	POST	`/api/auth/register`	Public	Creates a user and automatically opens a Basic Wallet with a zero balance.
Auth	POST	`/api/auth/login`	Public	Validates email and password, then returns a JWT. Typical responses include 200 OK or 401 Unauthorized.
Wallet	GET	`/api/wallets/my-wallet`	Authorized	Returns the logged-in user's wallet details and balance using `WalletResponseDto`.
Wallet	POST	`/api/wallets/deposit`	Authorized	Dummy top-up endpoint for development and testing.
Transfer	POST	`/api/transfers/p2p`	Authorized	Transfers funds between wallets. Use `DbContext.Database.BeginTransactionAsync()` to coordinate the database updates.
Statement	GET	`/api/transactions/history`	Authorized	Returns paginated history. Query parameters may include `pageNumber`, `pageSize`, `startDate`, and `type`. Use LINQ and `AsNoTracking()` for read-only queries.
Admin	PATCH	`/api/admin/wallets/{id}/freeze`	Admin only	Freezes or unfreezes a wallet. Requires role-based authorization such as `[Authorize(Roles="Admin")]`.

2.4 Mapping to the Master Technical Checklist

1. C# and Object-Oriented Programming (OOP)

- `WalletBase` will be an abstract class, with `BasicWallet` and `MerchantWallet` as derived types. Polymorphism will be demonstrated by overriding `CalculateFee()`.
- The `Balance` property will use a private setter (for example, `private set`) to enforce encapsulation.
- Use C# `record` types for DTOs, for example: `TransferRequestDto(string ReceiverWalletNumber, decimal Amount)`.

2. Interfaces, Dependency Injection, and Loose Coupling

- Define abstractions such as `ITransferService`, `IWalletRepository`, and `INotificationService`.
- Register implementations in `Program.cs`, for example: `builder.Services.AddScoped<ITransferService, TransferService>()`.

3. Generics, Delegates, and LINQ

- Create a generic API response wrapper, `ApiResponse<T>`, so endpoints can return a consistent JSON response structure.
- Use LINQ in transaction history queries: `Where()`, `OrderByDescending()`, `Skip()`, `Take()`, and `Select()` for filtering, sorting, pagination, and DTO projection.

4. Async/Await and Exception Handling

- Controller and repository operations should use asynchronous methods, such as `async Task<ActionResult<T>>`, where appropriate.
- Implement custom exceptions such as `InsufficientFundsException` and `WalletFrozenException`.
- Global exception-handling middleware should convert known application errors into consistent, safe HTTP error responses, such as a 400 Bad Request JSON response where appropriate.

5. Unit Testing (xUnit and Moq)

Create a separate `FinPay.Tests` project and test `TransferService` without connecting to a real database, using Moq to mock its dependencies.

Test	Expected Result
1. Insufficient funds	If the transfer amount exceeds the sender's available balance, the service throws `InsufficientFundsException`.
2. Self-transfer	If sender and receiver wallet numbers are identical, the transfer is rejected.
3. Frozen sender wallet	If the sender wallet has `IsFrozen == true`, the transfer is blocked.
4. Valid transfer	The sender's balance decreases and the receiver's balance increases by the correct amount.
5. Merchant fee calculation	`MerchantWallet.CalculateFee()` applies the configured percentage correctly.

Scope note: this SRS translates the supplied requirements into English while preserving the proposed project structure, schema, endpoints, technical concepts, and five initial unit tests.